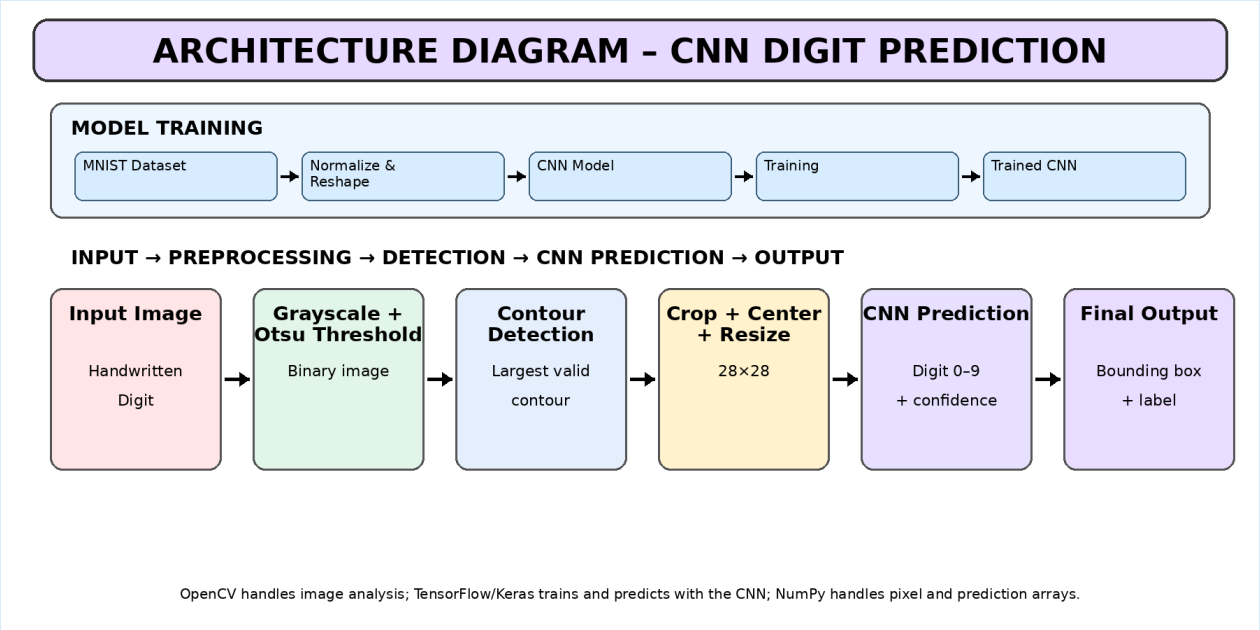
EX.NO: DATE:	MINI PROJECT - 1 CNN DIGIT PREDICTION (IMAGE ANALYSIS)

METHODOLOGY

The system uses image processing and a Convolutional Neural Network (CNN) to recognize handwritten digits. First, the MNIST dataset is loaded using TensorFlow and the image pixel values are normalized to the range 0–1. A CNN model is built using two convolution layers, max-pooling layers, a fully connected Dense layer, and a 10-class Softmax output layer. The model is trained for 5 epochs with batch size 64 and a 10% validation split. For a new input image, OpenCV converts the image to grayscale, applies Otsu thresholding, detects the largest valid contour, crops the digit, places it in a square canvas, resizes it to 28 × 28 pixels, and normalizes it. The trained CNN predicts the digit and confidence, and the program displays the enlarged input image with a bounding box and prediction label.

ARCHITECTURE



Architecture of CNN Digit Prediction System

TECHNOLOGIES / COMPONENTS

1. Python – Core programming language used to build the digit prediction system.

2. TensorFlow / Keras – Loads MNIST, builds the CNN architecture, trains the model, evaluates it, and performs prediction.

3. OpenCV (cv2) – Reads the input image, converts it to grayscale, applies thresholding, detects contours, resizes the display, draws the bounding box, and shows the result.

4. NumPy – Handles numerical arrays, pixel data, prediction values, and the final predicted class.

5. MNIST Dataset – Provides 60,000 training images and 10,000 test images of handwritten digits for CNN learning and evaluation.

6. CNN Model – Learns image features through convolution and pooling layers and classifies the digit into one of 10 classes.

7. Image Preprocessing – Converts the input to grayscale, thresholds it, extracts the largest contour, centers the digit, resizes it to 28 × 28, and normalizes pixel values.

8. Output Display – Shows the enlarged input image with a green bounding box and a label containing the predicted digit name and confidence.

CONCEPTS USED

CNN (Convolutional Neural Network) – A neural network designed for image data. It learns visual patterns using convolution filters and uses the learned features to classify digits.

Convolution Layer – Extracts local image features such as edges and shapes. The program uses Conv2D layers with 32 and 64 filters.

ReLU Activation – Introduces non-linearity so the network can learn complex patterns.

Max Pooling – Reduces the spatial size of feature maps while retaining important features.

Flatten Layer – Converts the extracted feature maps into a one-dimensional vector before classification.

Dense Layer – Uses the learned features for final classification; the program uses a Dense layer with 128 neurons.

Softmax – Produces probabilities for the 10 digit classes from 0 to 9.

Adam Optimizer – Optimizes the CNN weights during training.

Sparse Categorical Cross-Entropy – Loss function used because the MNIST labels are integer class values.

Contour Detection – Finds the boundary of the handwritten digit so the program can isolate the main digit from the input image.

Otsu Thresholding – Automatically chooses a threshold to separate the digit from the background.

Confidence – The highest probability returned by the CNN, expressed as a percentage.

OUTPUT

The program output is displayed in a resizable window named “Handwritten Digit Analysis”. The output image is enlarged by a scale factor of 15 and includes a bounding box around the detected digit together with the predicted digit name and confidence percentage. The actual screenshot below should be replaced with the screenshot produced by running the submitted program with img_47.jpg.

OUTPUT 1 – ACTUAL PROGRAM OUTPUT



Predicted Digit: EIGHT | Confidence: 97.52%

Green border indicates the detected digit region and the label shows the CNN prediction with confidence.

Output 1 – CNN predicted digit with confidence and bounding box

RESULT

Actual test output shown in the screenshot: EIGHT with 97.52% confidence.

The program successfully loads the MNIST dataset, normalizes the image data, builds and trains the CNN model, and evaluates the model on the test dataset. For the user-provided input image, OpenCV performs grayscale conversion, Otsu thresholding, contour detection, cropping, centering, and resizing. The processed digit is passed to the trained CNN, which returns the predicted digit and confidence. The final display enlarges the original image and marks the detected digit with a bounding box and prediction label. The exact predicted digit and confidence depend on the input file img_47.jpg and the trained model run.

CONCLUSION

Thus, the CNN Digit Prediction system was successfully designed using Python, TensorFlow/Keras, OpenCV, NumPy, and the MNIST dataset. The system combines image preprocessing with CNN-based classification to identify handwritten digits. It also provides a visual analysis result by displaying the input image with a bounding box, predicted digit name, and confidence percentage.

PROGRAM CONCEPT / WORKFLOW

- 1. Load MNIST dataset using TensorFlow/Keras.
- 2. Normalize pixel values and reshape images to 28 × 28 × 1.
- 3. Build CNN using Conv2D, MaxPooling2D, Flatten, Dense, and Softmax layers.
- 4. Compile the model using Adam optimizer and sparse categorical cross-entropy.
- 5. Train the CNN for 5 epochs with batch size 64 and 10% validation split.
- 6. Evaluate the trained model on the MNIST test dataset.
- 7. Read img_47.jpg using OpenCV and check that the image exists.
- 8. Convert the image to grayscale and apply Otsu binary thresholding.
- 9. Detect contours and select the largest valid contour as the digit.
- 10. Crop, center, resize to 28 × 28, normalize, and reshape the digit.
- 11. Predict the digit using the CNN and calculate confidence.
- 12. Enlarge the original image, draw the bounding box, add the prediction label, and display the result.